

Introduction to N-gram Language Models

Language Models (LMs)

- A language model (LM) is a machine learning model that predicts next upcoming words.
- More formally:
 - An LM assigns a probability to each potential next word
 - gives a probability distribution over possible next words
 - An LM can assign a probability to a whole sentence

Why word prediction?

- It's a helpful part of language tasks:
 - Autocomplete
 - Grammar or spell checking
 - Speech recognition
 - Machine translation
- It's how LLMs (Large Language Models) Work:
 - LLMs are **trained** to predict words.
 - Left-to-right (autoregressive) LMs learn to predict the next word.
 - LLMs generate text by predicting words
 - By predicting the next word over and over again

General Language Modeling

- **Goal:** Compute the probability of a sentence W or sequence of words $w_1, w_2, w_3, w_4, w_5 \dots w_n$:

$$P(W) = P(w_1, w_2, w_3, w_4, w_5, \dots, w_n) = P(w_{1:n})$$

- **OR** probability of an upcoming next word:

$$P(w_n | w_1, w_2, \dots, w_{n-1}) = P(w_n | w_{1:n-1})$$

- An LM computes either of these:

$$P(W) \text{ (i.e. } P(w_{1:n}))$$

$$\text{OR } P(w_n | w_1, w_2, \dots, w_{n-1}) \text{ (i.e. } P(w_n | w_{1:n-1}))$$

How to compute $P(W)$ or $P(w_n | w_1, \dots, w_{n-1})$?

- The joint probability $P(W) = P(w_1, w_2, w_3, w_4, w_5 \dots w_n) = P(w_{1:n})$
Rely on the Chain Rule of Probability
- **Reminder: The Chain Rule**
- Recall the definition of conditional probabilities
 $P(B|A) = P(A,B)/P(A) \rightarrow \text{Rewriting: } P(A,B) = P(A) P(B|A)$
- More variables:
 $P(A,B,C,D) = P(A) P(B|A) P(C|A,B) P(D|A,B,C)$
- The Chain Rule in General:
$$\begin{aligned} P(w_1, w_2, w_3, \dots, w_n) &= P(w_{1:n}) \\ &= P(w_1)P(w_2|w_1)P(w_3|w_1, w_2) \dots P(w_n|w_1, \dots, w_{n-1}) \\ &= P(w_1)P(w_2|w_1)P(w_3|w_{1:2}) \dots P(w_n|w_{1:n-1}) \end{aligned}$$

The Chain Rule applied to compute joint probability of words in sentence:

$$\begin{aligned} P(w_{1:n}) &= P(w_1)P(w_2|w_1)P(w_3|w_{1:2}) \dots P(w_n|w_{1:n-1}) \\ &= \prod_{k=1}^n P(w_k|w_{1:k-1}) \end{aligned}$$

$$\begin{aligned} P(\text{“The water of Walden Pond”}) &= \\ &P(\text{The}) \times P(\text{water}|\text{The}) \times P(\text{of}|\text{The water}) \\ &\times P(\text{Walden}|\text{The water of}) \times P(\text{Pond}|\text{The water of Walden}) \end{aligned}$$

The Problem: Long conditioning histories become impossible to compute because sequences never repeat identically in finite text.

Solution: Markov Approximation

Approximate context by looking back only $N - 1$ words:

- **Unigram ($N = 1$) Model:** $P(W) = P(w_1, \dots, w_n) \approx \prod_{k=1}^n P(w_k)$
- **Bigram ($N = 2$) Model:** $P(W) = P(w_1, \dots, w_n) \approx \prod_{k=1}^n P(w_k | w_{k-1})$
- **Trigram ($N = 3$) Model:** $P(W) = P(w_1, \dots, w_n) \approx \prod_{k=1}^n P(w_k | w_{k-2}, w_{k-1})$
- **General N-gram model:**

Approximate context by looking back only $N - 1$ words:

$$P(w_n | w_{1:n-1}) \approx P(w_n | w_{n-N+1:n-1})$$

N-gram ($N = N$) Language Model: $P(W) = P(w_1, \dots, w_n) \approx \prod_{k=1}^n P(w_k | w_{k-N+1:k-1})$

Language Models: N-grams

- N-gram language model predicts the probability of the next word given (N-1) previous words:
 - $N=1 \rightarrow$ Unigram
 - $N=2 \rightarrow$ Bigram
 - $N=3 \rightarrow$ Trigram
- Example:
 - "I love natural language processing"
- Unigrams ($N=1$):
 - ["I", "love", "natural", "language", "processing"]
- Bigrams ($N=2$):
 - ["I love", "love natural", "natural language", "language processing"]
- Trigrams ($N=3$):
 - ["I love natural", "love natural language", "natural language processing"]

Estimating probabilities using Maximum Likelihood Estimation (MLE) :

- Calculate relative frequency counts from a training corpus:
- **Unigram ($N = 1$):**
 - $P_{MLE}(w_k) = \frac{C(w_k)}{N_{total}}$
 - (where N_{total} is the total number of tokens in the corpus).
- **Bigram ($N = 2$):**
 - $P_{MLE}(w_k | w_{k-1}) = \frac{C(w_{k-1}, w_k)}{C(w_{k-1})}$
- **Trigram ($N = 3$):**
 - $P_{MLE}(w_k | w_{k-2}, w_{k-1}) = \frac{C(w_{k-2}, w_{k-1}, w_k)}{C(w_{k-2}, w_{k-1})}$
- **N-gram ($N = N$):**
 - $P_{MLE}(w_k | w_{k-N+1:k-1}) = \frac{C(w_{k-N+1:k-1}, w_k)}{C(w_{k-N+1:k-1})} = \frac{C(w_{k-N+1:k})}{\sum_w C(w_{k-N+1:k-1}, w)}$
- **What about start of sentence and end of sentence (sentence boundaries)?**
- Augment the sequence with sentence-boundary markers using special symbols e.g., $\langle s \rangle \rightarrow$ start of sentence and $\langle /s \rangle \rightarrow$ end of sentence,; and then apply all the calculations up to chain rule.

An example:

- **Training Corpus for Bigram Model:**

- $\langle s \rangle$ I am Sam $\langle /s \rangle$
- $\langle s \rangle$ Sam I am $\langle /s \rangle$
- $\langle s \rangle$ I do not like green eggs $\langle /s \rangle$

- **Calculate:** $P(\text{am} \mid \text{I})$

- $\text{Count}(\text{I}) = 3$
- $\text{Count}(\text{I}, \text{am}) = 2$
- $P(\text{am} \mid \text{I}) = \frac{2}{3}$

- **Calculate:** $P(\text{Sam} \mid \langle s \rangle)$

- $\text{Count}(\langle s \rangle) = 3$
- $\text{Count}(\langle s \rangle, \text{Sam}) = 1$
- $P(\text{Sam} \mid \langle s \rangle) = \frac{1}{3}$

Estimating probabilities using Maximum Likelihood Estimation (MLE) :

- To calculate the probability of an entire sentence $W = (w_1, w_2, \dots, w_n)$, augment the sequence with sentence-boundary markers (e.g., $\langle s \rangle$ and $\langle /s \rangle$) and apply the chain rule:
 - $P(W) = P(w_1, w_2, \dots, w_n) = \prod_{k=1}^{n+1} P_{MLE}(w_k | w_{k-N+1:k-1})$
 - Here $n+1^{\text{th}}$ term includes $\langle /s \rangle$ token Also, use $\langle s \rangle$ for $w_0, w_{-1}, \dots$
- In practice, compute this in **log-space** to prevent numerical underflow:
 - $-\log P(W) = \sum_{k=1}^{n+1} \log P_{MLE}(w_k | w_{k-N+1:k-1})$

Larger N-grams: Dataset?

- Large datasets of large N-grams have been released
 - Corpus of Contemporary American English (COCA) N-grams: 1 billion words: 1-grams up to 5-grams
 - Google Books N -gram: Max Order- 5-gram
 - Google Web 1T: (1 trillion words) Max Order- 5-gram
- **Newest model:** infini-gram (∞ -gram) (Liu et al 2024):
 - An open-source engine and framework introduced that allows computing n -gram language model probabilities and exact counts for **arbitrary N (from $N = 1$ up to $n = \infty$)** over trillion-token corpora in milliseconds.
 - No precomputing! Instead, store 5 trillion words of web text in **suffix arrays**.

Problems with N-gram models

1. N-grams can't handle **long-term dependencies**:

“**The soups** that I made from that new cookbook I bought yesterday **were** amazingly delicious.”

2. N-grams don't do well at modeling new sequences (even if they have similar meanings to sequences, they've seen)

→ **The solution: Large language models (LLMs)**

- Can handle much longer contexts because they use neural **embedding spaces**, can model meaning better

Why study N-gram models?

- They are the "fruit fly of NLP" - a simple model that introduces important topics for **large language models (LLMs)**
 - **training** and **testing** datasets
 - the **perplexity** performance metric
 - **sampling** to generate sentences
 - ideas like **interpolation** and **backoff**
 - N-grams are efficient and fast and useful in situations where LLMs are too heavyweight

How to Evaluate Performance of N-gram Language Models ?

How to evaluate performance of N-gram models?

- Evaluating the performance of an N -gram language model is done via two primary methodologies:
 - **Extrinsic evaluation** (measuring downstream task improvement)
 - **Intrinsic evaluation** (measuring the model's predictive power on unseen text)

"Extrinsic (in-vivo) Evaluation"

To compare models A and B

1. Put each model in a real task
 - Machine Translation, speech recognition, etc.
 2. Run the task, get accuracy score for A and for B:
 - How many words translated correctly
 - How many words transcribed correctly
 3. Compare accuracy for A and B
- Extrinsic evaluation not always possible
 - Expensive, time-consuming
 - Doesn't always generalize to other applications

Intrinsic (in-vitro) evaluation

- **Perplexity:**

- An **evaluation metric** (like **perplexity**) tells us how well our model does on the test set.
- Directly measures language model performance at predicting words.
- Doesn't necessarily correspond with real application performance but gives us a single general metric for language models
- Useful for large language models (LLMs) as well as N-grams
- Perplexity is directly derived from **Claude Shannon's** foundational work in **Information Theory (1948)** and his 1951 paper, *"Prediction and Entropy of Printed English."*
- Perplexity translates Shannon's abstract measure of information (bits) into an intuitive, physical scale representing word-choice uncertainty.



Claude Shannon

Intrinsic (in-vitro) evaluation

- **Perplexity:**

- Intrinsic evaluation tests how well the model predicts a held-out test corpus $W = (w_1, w_2, \dots, w_n)$ without running a downstream application. Here the last word includes </s>
- **Perplexity (PP)** is the standard intrinsic metric.
- It represents the inverse probability of the test set, normalized by the total number of words n :
- $$PP(W) = P(W)^{-\frac{1}{n}} = P(w_1, w_2, \dots, w_n)^{-\frac{1}{n}} = \sqrt[n]{\prod_{k=1}^n \frac{1}{P(w_k | w_{k-N+1:k-1})}}$$
- **Interpretation:** Perplexity can be thought of as the **effective branching factor**—the average number of equally likely words the model is choosing among at each step.
- **Goal: Lower perplexity indicates a better model.** A lower perplexity means the model assigns higher probabilities to the actual words that appeared in the test set.
- Probability range is $[0,1]$, perplexity range is $[1,\infty]$
- **Minimizing perplexity is the same as maximizing probability**

Intrinsic (in-vitro) evaluation

- **Log Perplexity / Cross-Entropy:**

- Because calculating product probabilities leads to numerical underflow, perplexity is computed in log-space via average negative log-likelihood (Cross-Entropy $H(W)$):

- $$H(W) = -\frac{1}{n} \sum_{k=1}^n \log_2 P(w_k \mid w_{k-N+1:k-1})$$

- $$PP(W) = 2^{H(W)}$$

- **Critical Requirements for Fair Perplexity Comparison**

- **Identical Vocabulary:** Models must share the exact same vocabulary (including the <UNK> token handling). Comparing perplexities across models with different vocabularies is mathematically invalid because an open vocabulary with high unknown-token assignment artificially deflates perplexity.
- **Identical Tokenization & Test Set:** Both models must be evaluated on the exact same tokenized test split with identical sentence-boundary markers (<s>, </s>).

Example:

- **Training Corpus for Bigram Model:**

- `<s> I am Sam </s>`
- `<s> Sam I am </s>`
- `<s> I do not like green eggs </s>`

- **Test Sequence**

- `<s> I am Sam </s>`

- **Estimate probability of Test sequence using MLE.**
- **Find Perplexity score using direct formula and using Cross Entropy.**
- **Repeat the task for Trigram model.**
- **Interpret the results.**

Solution:

- Model: $N = 2$ (Bigram)
- $P(W) = P(l \mid \langle s \rangle) \times P(\text{am} \mid l) \times P(\text{Sam} \mid \text{am}) \times P(\langle /s \rangle \mid \text{Sam})$
- Test data: $n = 4$ considering one more than 3 actual tokens for sentence boundary
- Evaluated tokens: $w_1 = l$, $w_2 = \text{am}$, $w_3 = \text{Sam}$, $w_4 = \langle /s \rangle$
- Number of tokens: $n = 4$
- **Relevant Counts:**
- $C(\langle s \rangle) = 3$
- $C(l) = 3$
- $C(\text{am}) = 2$
- $C(\text{Sam}) = 2$
- $C(\langle s \rangle, l) = 2$
- $C(l, \text{am}) = 2$
- $C(\text{am}, \text{Sam}) = 1$
- $C(\text{Sam}, \langle /s \rangle) = 1$

• **Step 1:** $P(l \mid \langle s \rangle)$

$$P(l \mid \langle s \rangle) = \frac{c(\langle s \rangle, l)}{c(\langle s \rangle)} = \frac{2}{3} \approx 0.66667$$

• **Step 2:** $P(\text{am} \mid l)$

$$P(\text{am} \mid l) = \frac{c(l, \text{am})}{c(l)} = \frac{2}{3} \approx 0.66667$$

• **Step 3:** $P(\text{Sam} \mid \text{am})$

$$P(\text{Sam} \mid \text{am}) = \frac{c(\text{am}, \text{Sam})}{c(\text{am})} = \frac{1}{2} = 0.50000$$

• **Step 4:** $P(\langle /s \rangle \mid \text{Sam})$

$$P(\langle /s \rangle \mid \text{Sam}) = \frac{c(\text{Sam}, \langle /s \rangle)}{c(\text{Sam})} = \frac{1}{2} = 0.50000$$

$$P(W) = \frac{2}{3} \times \frac{2}{3} \times \frac{1}{2} \times \frac{1}{2} = \frac{4}{36} = \frac{1}{9} \approx 0.11111$$

$$PP(W) = P(W)^{-\frac{1}{n}} \text{ where } n = 4:$$

$$PP(W) = \left(\frac{1}{9}\right)^{-\frac{1}{4}} = 9^{\frac{1}{4}} = \sqrt{\sqrt{9}} = \sqrt{3} \approx 1.732$$

Solution:

- **Calculation via Log Cross-Entropy:**

- $\log_2 P(l | \langle s \rangle) = \log_2(2/3) \approx -0.58496$
- $\log_2 P(am | l) = \log_2(2/3) \approx -0.58496$
- $\log_2 P(\text{Sam} | am) = \log_2(1/2) = -1.00000$
- $\log_2 P(\langle /s \rangle | \text{Sam}) = \log_2(1/2) = -1.00000$

- **Cross-Entropy $H(W)$:**

- $$H(W) = -\frac{1}{4}((-0.58496) + (-0.58496) + (-1.00000) + (-1.00000)) = -\frac{1}{4}(-3.16992) \approx 0.79248 \text{ bits/word}$$

- **Perplexity:**

- $$PP(W) = 2^{H(W)} = 2^{0.79248} = \sqrt{3} \approx 1.732$$

Solution:

Model: $N = 3$ (Trigram)

Represent training data set for Trigram

Model:

$\langle s \rangle \langle s \rangle$ I am Sam $\langle /s \rangle$

$\langle s \rangle \langle s \rangle$ Sam I am $\langle /s \rangle$

$\langle s \rangle \langle s \rangle$ I do not like green eggs $\langle /s \rangle$

Test data:

$\langle s \rangle \langle s \rangle$ I am Sam $\langle /s \rangle$

Evaluated tokens: $w_1 = \text{I}$, $w_2 = \text{am}$, $w_3 = \text{Sam}$, $w_4 = \langle /s \rangle$

Number of tokens (n): 4

$$\begin{aligned} P(W) &= P(\text{I} \mid \langle s \rangle, \langle s \rangle) \times P(\text{am} \mid \langle s \rangle, \text{I}) \\ &\times P(\text{Sam} \mid \text{I}, \text{am}) \times P(\langle /s \rangle \mid \text{am}, \text{Sam}) \\ &= P(\text{I} \mid \langle s \rangle) \times P(\text{am} \mid \text{I}) \times P(\text{Sam} \mid \text{am}) \\ &\times P(\langle /s \rangle \mid \text{Sam}) \end{aligned}$$

The Trigram MLE formula is:

$$P_{MLE}(w_k \mid w_{k-2}, w_{k-1}) = \frac{C(w_{k-2}, w_{k-1}, w_k)}{C(w_{k-2}, w_{k-1})}$$

- **Step 1:** $P(\text{I} \mid \langle s \rangle, \langle s \rangle)$
- $P(\text{I} \mid \langle s \rangle, \langle s \rangle) = \frac{c(\langle s \rangle, \langle s \rangle, \text{I})}{c(\langle s \rangle, \langle s \rangle)} = \frac{2}{3} \approx 0.66667$
- **Step 2:** $P(\text{am} \mid \langle s \rangle, \text{I})$
- $P(\text{am} \mid \langle s \rangle, \text{I}) = \frac{c(\langle s \rangle, \text{I}, \text{am})}{c(\langle s \rangle, \text{I})} = \frac{1}{2} = 0.50000$
- **Step 3:** $P(\text{Sam} \mid \text{I}, \text{am})$
- $P(\text{Sam} \mid \text{I}, \text{am}) = \frac{c(\text{I}, \text{am}, \text{Sam})}{c(\text{I}, \text{am})} = \frac{1}{2} = 0.50000$
- **Step 4:** $P(\langle /s \rangle \mid \text{am}, \text{Sam})$
- $P(\langle /s \rangle \mid \text{am}, \text{Sam}) = \frac{c(\text{am}, \text{Sam}, \langle /s \rangle)}{c(\text{am}, \text{Sam})} = \frac{1}{1} = 1.00000$
- $P(W) = \frac{2}{3} \times \frac{1}{2} \times \frac{1}{2} \times 1 = \frac{2}{12} = \frac{1}{6} \approx 0.16667$
- $PP(W) = \left(\frac{1}{6}\right)^{-\frac{1}{4}} = 6^{\frac{1}{4}} = \sqrt{\sqrt{6}} \approx \sqrt{2.44949} \approx 1.565$

Solution:

Calculation via Log Cross-Entropy:

$$\log_2 P(l | \langle s \rangle, \langle s \rangle) = \log_2(2/3) \approx -0.58496$$

$$\log_2 P(am | \langle s \rangle, l) = \log_2(1/2) = -1.00000$$

$$\log_2 P(Sam | l, am) = \log_2(1/2) = -1.00000$$

$$\log_2 P(\langle /s \rangle | am, Sam) = \log_2(1) = 0.00000$$

Cross-Entropy $H(W)$:

$$\begin{aligned} H(W) &= -\frac{1}{4}((-0.58496) + (-1.00000) + (-1.00000) + 0.00000) \\ &= -\frac{1}{4}(-2.58496) \approx 0.64624 \text{ bits/word} \end{aligned}$$

Perplexity:

$$PP(W) = 2^{H(W)} = 2^{0.64624} \approx 1.565$$

The trigram model achieves a lower perplexity (**1.565** vs. **1.732** for the bigram model) because the additional word of history resolves ambiguity (such as knowing with certainty that $\langle /s \rangle$ follows $am\ Sam$)

Holding test set constant: Lower perplexity = better language model